

Pandas

一、数据的输入与输出 (I/O)

1. 读取数据 (`read_...`)

将外部文件加载为 `DataFrame` (二维数据结构) 或 `Series` (一维数据结构) :

- **通用文件**: `read_csv` (最常用), `read_excel`, `read_json`, `read_html` (网页表格)。
- **数据库/大数据**: `read_sql` (数据库), `read_hdf` (HDF5), `read_gbq` (实验性)。
- **统计软件格式**: `read_stata`, `read_sas`, `read_spss` ()。
- **其他**: `read_pickle`, `read_msgpack`, `read_clipboard` 。

2. 保存数据 (`to_...`)

将 `DataFrame` 导出为文件:

- **对应读取**: `to_csv`, `to_excel`, `to_json`, `to_html`, `to_sql`, `to_hdf` 等。
- **注意**: 保存时通常可以指定 `index=False` 来避免保存自动生成的行索引。

二、核心数据结构与属性

1. 三大核心对象

- **Series**: 一维数组, 带标签 (Index) 。
- **DataFrame**: 二维表格, 由多个 `Series` 组成, 有行索引 (Index) 和列名 (Columns) 。
- **Timestamp**: 时间戳对象, 用于处理时间序列数据。

2. 数据类型与属性查看

- `.dtypes`
: 查看每一列的数据类型 (如

```
int64
float64
object
...
```

◦ 注: `dtype='object'` 通常代表字符串或混合类型。

- `.values`: 获取底层的 NumPy 数组形式 (不带索引和列名) 。

- `.index / .columns`: 获取行索引名称或列名列表。
 - `.describe()`: 快速生成数值列的统计摘要 (计数、均值、标准差、最小/大值、分位数)。
 - `.T`: 转置操作 (行列互换)。
-

三、数据索引与选择 (Selection)

1. 基础切片与选择

- `df[列名]`: 选择单列 (返回 Series)。
- `df[0:3]`: 行切片。注意是左闭右开 (取第0、1、2行), 类似 Python 列表切片。
- 布尔索引 (): `df[df.A > 8]`, 筛选出 A 列大于 8 的所有行。

2. 高级索引器 (Loc & iLoc)

- `.loc[标签]`: 基于标签 (Label)选择。
 - 特点: 切片时是左闭右闭 (包含结束标签)。
- `.iloc[m, n]`: 基于整数位置 (Integer Position)选择。
 - 用法: `m` 是行号, `n` 是列号 (从0开始)。切片时左闭右开。

3. 其他选择方式

- `.ix`: 混合选择 (已废弃/Deprecated)。早期版本允许混用标签和整数, 现在建议严格使用 `.loc` 或 `.iloc`。
 - Select by local: 通常指使用 `query()` 方法或局部变量进行条件筛选。
-

四、数据清洗与预处理

处理“脏”数据, 使其变得可用。

1. 缺失值处理 (NaN)

- `.isnull()`: 检测缺失值, 返回布尔矩阵 (True 表示缺失)。
- `.dropna(axis=0, how='any')`
 - : 删除缺失值。
 - `axis=0`: 按行删 (默认); `axis=1`: 按列删。
 - `how='any'`: 只要有 NaN 就删; `how='all'`: 全为 NaN 才删。
- `.fillna(value=0)`: 填充缺失值。可以用常数 (如0)、均值或前向填充等方法。

2. 排序

- `.sort_values(by=...)`: 按数值大小排序。 `by` 指定依据哪一列。
- `.sort_index(axis=1, ascending=False)`: 按索引标签排序。
 - `axis=1` 表示对列名进行排序; `ascending=False` 表示降序。

3. 其他处理

- **Categorical**: 分类数据类型, 适用于取值有限且重复率高的列 (如性别、城市), 可节省内存并优化性能。
- **Periods**: 时间段对象, 用于表示如“2023年1月”这样的时间段。

五、数据合并与重塑 (Merging)

将多张表拼在一起, 类似于 SQL 的 Join 操作。

1. 拼接 (Stacking)

- `pd.concat([df1, df2], axis=0, ignore_index=True)`:
 - 物理拼接。 `axis=0` 是上下堆叠 (增加行数), `axis=1` 是左右拼接 (增加列数)。
 - `ignore_index=True`: 重置索引, 忽略原来的行号。

2. 关联合并 (Join/Merge)

- `pd.merge(left, right, on='key')`: 基于共同的列 (键) 进行数据库风格的合并。
 - `on`: 指定连接键 (可以是单列或多列 `['key1', 'key2']`)。
 - `how`:
 - `'inner'` (默认): 取交集 (只保留两边都有的)。
 - `'outer'`: 取并集 (没有的补 NaN)。
 - `'left'` / `'right'`: 以左表或右表为主。
 - `indicator=True`: 增加一列 `_merge`, 显示数据来源 (`left_only`, `right_only`, `both`)。
 - `suffixes=['_x', '_y']`: 当两表有非连接键的同名列时, 用于区分后缀。

3. 旧版方法

- `.append`: 在 DataFrame 末尾追加行 (已废弃, 建议使用 `concat`)。
- `join_axes`: 在旧版 `merge/join` 中强制指定索引对齐方式。

六、数据可视化 (Plotting)

Pandas 内置了基于 Matplotlib 的快捷绘图接口 `df.plot()`。

- **bar**: 柱状图 (适合分类对比)。

- **hist**: 直方图 (适合看分布)。
 - **box**: 箱线图 (适合看异常值和四分位)。
 - **kde**: 核密度估计图 (平滑的分布曲线)。
 - **area**: 面积图。
 - **scatter**: 散点图 (适合看相关性, 需指定 x, y)。
 - **hexbin**: 六边形分箱图 (适合大数据量的散点分布)。
 - **pie**: 饼图 (适合看占比)。
-

💡 旧版本的写法 (如 `.ix` 和 `.append`) , 后使用 `.loc/.iloc` 和 `pd.concat`